AI-Powered Document De-Duplication and Consolidation for Efficient Information Management

by
Maria Lisa D Silva

I. Project Description:

This project seeks to develop an AI-driven solution to manage and organize large collections of documents, specifically targeting duplicate and similar content in .docx files. Utilizing advanced machine learning (ML) and natural language processing (NLP) techniques, the project will cluster documents based on content and subject matter, leading to the generation of one unique consolidated document for each cluster, ensuring that all relevant information is included without duplicates.

II. Aim:

To develop an AI-driven solution for efficient management and organization of a vast collection of documents, specifically targeting the issue of duplicate and similar content.

III. Objective:

i. Create an AI system to manage and organize a large collection of .docx documents efficiently.
ii. Implement algorithms to cluster documents based on content and subject matter.
iii. Identify and eliminate duplicate and similar information within each document cluster.
iv. Generate a single, comprehensive document that encompasses all relevant information from each cluster.
v. Design the system to be scalable to handle documents from various domains and industries.

IV. Logic:

i. Read .docx files from a specified directory. Remove stopwords and punctuations from the text. Lemmatize the text to reduce words to their base form.

ii. Vectorize the preprocessed text using the TF-IDF Vectorizer. Assign numerical values to words based on their frequency in the document and rarity in the corpus.

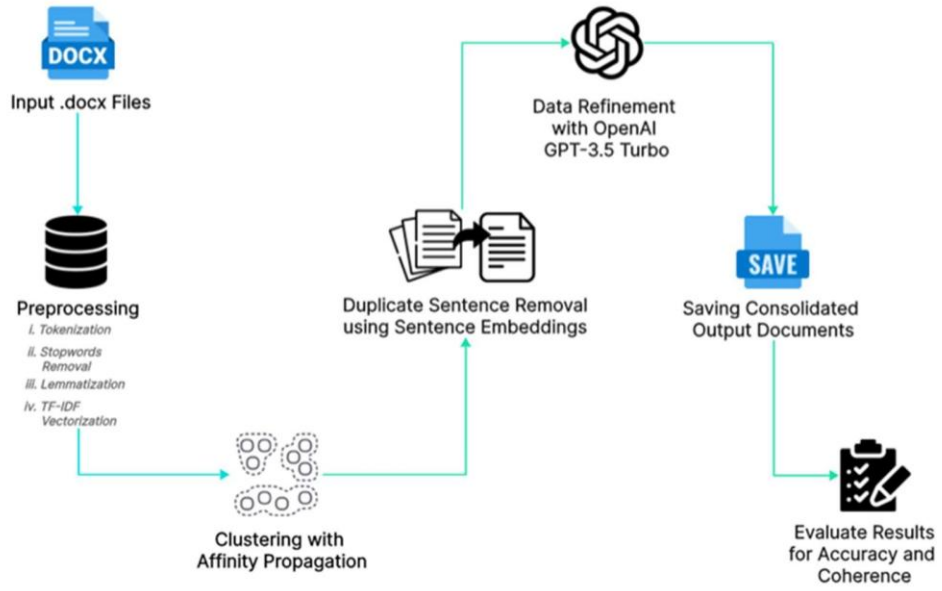
iii. Perform clustering using the Affinity Propagation algorithm. Automatically determine the number of clusters based on the input data. Group similar documents together based on their vector representations.

iv. Merge documents within each cluster to create consolidated representations. Identify and remove duplicates by calculating cosine similarity between document pairs. If two documents have high similarity, consider one as a duplicate and remove it.

v. Send the merged and duplicate-free documents to the GPT-3.5-turbo model for further refinement. Utilize GPT-3.5-turbo for tasks like text correction and title extraction. Divide the text into chunks of 4000 tokens to manage token limits using tiktoken. Save the corrected and processed documents in .docx format in an output directory.

vi. Calculate the similarity score between the input and output documents and analyze the lengths of the input and output documents.

AI-Powered Document De-Duplication and Consolidation for Efficient Information Management



v. Why This Approach?

i. NLTK Library:

NLTK (Natural Language Toolkit) is a powerful Python library for text processing, supporting tokenization, stemming, lemmatization, and stop-word removal. It's welldocumented, widely used, and reliable for text preprocessing tasks.

ii. TF-IDF Vectorizer:

The TF-IDF vectorizer converts text to numerical vectors, highlighting word importance relative to a document collection. It reduces the impact of common words, enhancing text representation quality for clustering.

iii. Affinity Propagation:

Affinity Propagation is a clustering algorithm that identifies exemplars without needing a predefined number of clusters. It effectively groups similar documents, making it ideal when cluster numbers are unknown.

iv. Cosine Similarity:

Cosine similarity measures the angle between document vectors, effectively assessing similarity regardless of length. It helps identify and remove duplicates within clusters, ensuring the final merged document is concise and unique.

v. GPT-3.5-turbo Model:

GPT-3.5-turbo, a state-of-the-art language model, corrects grammar and punctuation in merged documents, ensuring polished output. It also extracts meaningful titles for clusters, enhancing organization and readability.

vi. Evaluation Metrics:

Calculating similarity scores between input and output documents assesses information preservation. Analyzing document length reduction ensures effective consolidation, making the digital workspace more efficient without losing essential content.

VI. Implementation:

1. Data Collection:

   Manually gathered 50+ '.docx' files from diverse industries including technology (e.g., Amazon, Oracle), pharmaceutical (e.g., SunPharma, Glenmark, WebMD), automobile (e.g., Toyota, BMW, Volkswagen), food & beverages (e.g., Nestle, Coca-Cola) and retail (e.g., Walmart) to provide a comprehensive dataset.

2. Library Imports and Loading Environment File:

   - SentenceTransformers: Used for creating high-quality sentence embeddings for clustering and semantic similarity tasks.
   - nltk: A comprehensive library for natural language processing tasks.
   - docx2txt: Extracts text from '.docx' files for processing.
   - scipy.spatial.distance.cosine: Computes cosine distance between vectors for similarity assessment.
   - openai: Accesses OpenAI's GPT-3.5-turbo API for text generation and correction.
   - docx.Document: Creates and modifies '.docx' files.
   - dotenv.load_dotenv: Loads environment variables from a '.env' file.
   - tabulate: Formats data in tables for better readability.
   - tiktoken: Tokenizes text data for efficient processing.

   Environment File:
   An .env file is used to store and manage sensitive configuration details and environment variables securely.
   .env file is loaded using command "load_dotenv" and Input Folder Path, Output Folder Path and API Key is fetched.

3. Applying NLP Techniques for Preprocessing & Reading Documents:

   i. Text Preprocessing: The `preprocess` function cleans text by removing punctuation, tokenizing, eliminating stopwords, and lemmatizing words to lowercase. It rejoins words into a string, printing errors if exceptions occur.

   ii. Reading Documents: The `read_data` function reads `.docx` files from a directory, storing raw content, processed content, and filenames in lists. It uses `docx2txt` and `preprocess`, returning these lists.

4. Creating vectors & Clustering Using Affinity Propagation:

i. Vectorization: The `vectors` function converts documents into TF-IDF vectors using `TfidfVectorizer` from `scikit-learn`. It fits and transforms the documents, printing errors if any occur and returning `None`.

ii. Applied Affinity Propagation: The `clusters` function uses `AffinityPropagation` from `scikit-learn` to cluster documents based on TF-IDF matrix `X`. It fits the model, predicts cluster labels, and prints errors if any, returning `None`.

5. Removing Duplicates & Generating Meaningful Content using ChatGPT 3.5 Turbo:

   i. Removing Duplicates: The `remove_duplicates` function eliminates duplicate sentences using `SentenceTransformer` embeddings and cosine similarity. Sentences exceeding a 0.70 similarity threshold are removed. Errors are printed, and `None` is returned if any occur.

   ii. Generating Meaningful content: The `correct_grammar` function uses OpenAI's GPT3.5 to improve text grammar. It splits text into chunks, sends them for correction, and reassembles them. Errors default to using original text.

6. Extracting Company Names for File Naming:

   The `recognize_title` function extracts company names from text using OpenAI's GPT-3.5. It constructs a prompt, sends it to the API, and returns the name, printing errors if any.

7. Saving Consolidated Files with Descriptive Titles:

   The `saving_file` function saves corrected texts to `.docx` files in a specified directory, creating it if needed. It processes each item, handles errors, and prints relevant error messages if any.

8. Evaluating Results using Cosine Similarity:

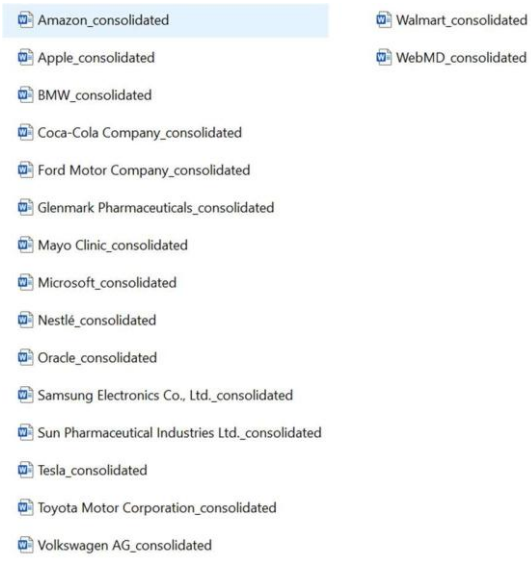
   The `calculate_cosine_similarity` function computes the cosine similarity between two text strings using TF-IDF vectorization, returning their similarity score. The `evaluation` function compares two lists of texts, `input_file` and `output_file`, by calculating similarity scores for corresponding pairs and their lengths. It ensures both lists are of equal length, raises an error if not, and then creates a DataFrame containing company names, similarity scores, and text lengths. If any errors occur, it prints an error message and returns `None`.

VII. Results:

i. Clusters: After processing 54 input DOCX files with the NLP library NLTK, vectors were created using TF-IDF vectorizer. Affinity propagation then grouped these files into 17 content-similar clusters.

```
Cluster 2:
Document 1: Amazon doc1.docx
Document 2: Amazon doc2.docx
Document 3: Amazon doc3.docx
Document 4: Amazon doc4.docx
Document 5: Amazon doc5.docx
Document 6: Doc_1.docx
Document 7: Doc_2.docx
Document 8: Doc_3.docx
Cluster 0:
Document 1: bmw1.docx
Document 2: bmw2.docx
Cluster 1:
Document 1: Coca1.docx
Document 2: Coca2.docx
Document 3: Coca3.docx
Document 4: Coca4.docx
Document 5: Coca5.docx
Cluster 3:
Document 1: Doc_4.docx
Document 2: Doc_5.docx
Document 3: Doc_6.docx
Cluster 4:
Document 1: Doc_7.docx
Document 2: Doc_8.docx
Document 3: Doc_9.docx
Cluster 5:
Document 1: ford.docx
Document 2: Ford2.docx
Cluster 6:
Document 1: Glenmark1.docx
Document 2: Glenmark2.docx
Cluster 7:
Document 1: MayoClinic1.docx
Document 2: MayoClinic2.docx
Document 3: MayoClinic3.docx
Cluster 8:
Document 1: Nestle1.docx
Document 2: Nestle2.docx
Cluster 9:
Document 1: Oracle-1 (1).docx
Document 2: Oracle-2.docx
Document 3: Oracle-3.docx
Document 4: Oracle-4.docx
Document 5: Oracle-5.docx
Cluster 10:
Document 1: Samsung doc1.docx
Document 2: Samsung doc2.docx
Document 3: Samsung doc3.docx
Document 4: Samsung doc4.docx
Document 5: Samsung doc5.docx
Cluster 11:
Document 1: SunPharma1.docx
Document 2: SunPharma2.docx
Document 3: SunPharma3.docx
Cluster 12:
Document 1: Tesla1.docx
Document 2: Tesla2.docx
Cluster 13:
Document 1: toyota.docx
Document 2: toyota2.docx
Cluster 14:
Document 1: volkswagen1.docx
Document 2: VolkswagenAG.docx
Cluster 15:
Document 1: Walmart1.docx
Document 2: Walmart2.docx
Document 3: Walmart3.docx
Cluster 16:
Document 1: WebMD1.docx
Document 2: WebMD2.docx
```

ii. Output Folder: After merging the clusters into individual documents, duplicates were removed using cosine similarity. The documents were then sent to OpenAI's GPT-3.5 Turbo for grammar and punctuation correction and title extraction. Each document was saved in .docx format, named as 'title_consolidated.docx', and stored in the output folder.



iii. Evaluation Metrics: The similarity between each input file and its coresponding consolidated output file was calculated to ensure the document's meaning remained unchanged. Additionally, lengths were analyzed to verify duplicate removal. All results were compiled into a DataFrame for detailed analysis.

Company Name	Similarity Score	Input Length	Output Length
Amazon	0.983837	59605	34221
BMW	0.998087	12214	11041
Coca-Cola Company	0.9944	30368	21331
Apple	0.990814	27556	20969
Microsoft	0.992377	32999	21237
Ford Motor Company	0.998013	27467	23675
Glenmark Pharmaceuticals	0.984228	31306	21358
Mayo Clinic	0.989924	56874	37469
Nestlé	0.98974	11814	8677
Oracle	0.98087	29938	19972
Samsung Electronics Co., Ltd.	0.988838	28861	18594
Sun Pharmaceutical Industries Ltd.	0.728795	46602	22729
Tesla	0.981244	11915	7764
Toyota Motor Corporation	0.984882	11955	7764
Volkswagen AG	0.988012	25786	17555
Walmart	0.984062	20266	12359
WebMD	0.992461	31548	23201

VIII. Guidelines:

i. Open your terminal and navigate to the directory where the application is located cd path/to/your/application
ii. Run the following command to install all the required libraries: pip install -r requirements.txt
iii. Create a .env file in the root directory of your application with the following content:

```
INPUT_FOLDER_PATH='path/to/your/input/folder'
OUTPUT_FOLDER_PATH='path/to/your/output/folder'
OPENAI_API_KEY='your-openai-api-key'
```

Replace path/to/your/input/folder, path/to/your/output/folder, and your-openai-api-key with your actual input folder path, output folder path, and OpenAI API key respectively.

iv. To run the application, execute the following command in your terminal: python Team56.py
v. Input File Requirements:
   a. All input files should be in .docx format.
   b. All input files should be kept in one folder.
   c. The folder path should be specified in the .env file under INPUT_FOLDER_PATH.

IX. Conclusion:

This project successfully developed an AI-driven solution for efficiently managing and organizing large collections of .docx documents, specifically targeting duplicate and similar content. By leveraging advanced NLP techniques and machine learning algorithms, we effectively clustered documents based on content and subject matter. Each cluster was merged into a unique, consolidated document, ensuring all relevant information was included without duplicates.

The process involved thorough text preprocessing using NLTK, TF-IDF vectorization, and clustering via Affinity Propagation. Duplicate removal was achieved using cosine similarity, and further text refinement, including grammar and punctuation correction, was performed using OpenAI's GPT-3.5 Turbo model. The system handled large text chunks efficiently with tiktoken, and the final corrected files were saved in .docx format.

Evaluation of the system showed that the meaning of the documents remained consistent, and duplicates were effectively removed. The results were compiled into a DataFrame, providing a detailed analysis of similarity scores and document lengths.